

Lab 2: ROS Nodes and Simulation Models

EEME E6911 — Probabilistic Robotics

Howard Wang | UNI: hw3214

Andres Permuy | UNI: app2164

Samwel Waweru | UNI: swk2139

Overview

This lab connects ROS 2 nodes and launch files with a Gazebo world containing a TurtleBot3 and a motorized glass door. A custom open-loop controller opens the door, drives the robot through, stops it, and closes the door. The forward speed is configurable at launch. Repeated trials compare commanded speed with the velocity reported by odometry.

Assignment 1 — Launch files

1. With default arguments, which first-level launch files are included, from which packages, and which are skipped?

Skipped: `flaky_door_opener_launch.py` (`prob_robot_labs`), because `run_door_opener` defaults to `false`,
`video_processor_launch.py` (`prob_robot_vision`) and `image_mean_feature_x_launch.py` (`prob_robot_labs`), because `run_vision_processor` defaults to `false`.

Included: `turtlebot3_world_launch.py` from package `prob_robot_labs`.

```
turtlebot3_world_launch_cmd = IncludeLaunchDescription(
    PythonLaunchDescriptionSource(
        os.path.join(get_package_share_directory('prob_robot_labs'),
                     'launch', 'turtlebot3_world_launch.py')
    ),
    launch_arguments={
        'world': world,
        'x_pose': x_pose,
        'y_pose': y_pose
    }.items()
)
```

2. Which launch file and package are included with `run_door_opener:=true`?

`flaky_door_opener_launch.py` from `prob_robot_labs`. It is included when `run_door_opener` is `true`, via `IfCondition(run_door_opener)`

```
run_door_opener = LaunchConfiguration('run_door_opener', default='false')

run_vision_processor = LaunchConfiguration('run_vision_processor', default='false')
max_vision_features = LaunchConfiguration('max_vision_features', default='200')
```

3. What is the default robot spawn position in the X–Y plane?

The robot spawns at $(x, y) = (-1.5, 0.0)$, from the defaults `x_pose=-1.5'` and `y_pose=0.0'`

```
def generate_launch_description():
    use_sim_time = LaunchConfiguration('use_sim_time', default='true')
```

```
world = LaunchConfiguration('world', default='door.world')
x_pose = LaunchConfiguration('x_pose', default='-1.5')
y_pose = LaunchConfiguration('y_pose', default='0.0')
```

4. What launch command spawns the robot at (-5, 1)?

`ros2 launch prob_rob_labs turtlebot3_and_door_launch.py x_pose:=-5 y_pose:=1`

Assignment 2 — Door model and ROS bridge

1. Which lines, in which launch file, instantiate the ROS–Gazebo bridge?

The bridge is created by `door_torque_bridge_cmd = Node(package='ros_gz_bridge', executable='parameter_bridge', )` and added to the launch description with `ld.add_action(door_torque_bridge_cmd)`

```
door_torque_bridge_cmd = Node(
    package='ros_gz_bridge',
    executable='parameter_bridge',
    name='door_torque_bridge',
    arguments=['/model/hinged_glass_door/joint/hinge/cmd_force@std_msgs/msg/Float64]gz.msgs.Double'],
    remappings=[
        ('/model/hinged_glass_door/joint/hinge/cmd_force', '/hinged_glass_door/torque')
    ],
```

2. What is the ROS bridge node's name? Support the answer with `ros2 node output`.

The node is `door_torque_bridge` (set by `name='door_torque_bridge'`).

3. What are the name and message type of the ROS topic controlling hinge torque?

Topic: `/hinged_glass_door/torque`; type: `std_msgs/msg/Float64`.

4. Which two links are connected by the hinge joint?

The two links connected by the hinge joint are `frame` and `door`.

```
<joint name="hinge" type="revolute">
  <parent>frame</parent>
  <child>door</child>
```

5. Which link moves, and what is its mass?

The moving link is `door`, and its mass is `41.3256` (kg, from `<inertial><mass>`).

```
<inertial>
<mass>41.3256</mass>
<pose>0.4569 0.0349 1.016 0 0 0</pose>
<inertia>
```

6. Why is the mass not specified for the other link?

`Frame` has no `<inertial>` block because it is welded to the world by `world_joint`, so it is static and doesn't need a mass.

7. What complete `ros2 topic pub` command successfully opened the door?

`ros2 topic pub --rate 10 /hinged_glass_door/torque std_msgs/msg/Float64 "{data: 1.0}"`

8. What minimum torque opened the door?

The minimum torque that opened the door was approximately `5 N·m`.

Assignment 3 — Open-loop door traversal

Implementation

The door_controller node uses a timer callback at 20 Hz and a timed state machine. It waits for subscribers on both command topics before beginning a startup delay. Stage timing uses the ROS simulation clock. The controller does not subscribe to camera, odometry, or door-state feedback; stage transitions depend only on elapsed time.

Published topic	Message / purpose
/cmd_vel	geometry_msgs/msg/TwistStamped; forward velocity
/hinged_glass_door/torque	std_msgs/msg/Float64; hinge torque in N m

Each velocity command has the current ROS timestamp and the base_footprint frame. Commands are published repeatedly to satisfy the supplied velocity-command timeout. Angular velocity and unused linear components remain zero.

Default sequence

Stage	Duration (s)	Speed (m/s)	Torque (N m)
STARTUP	3	0	0
OPENING	8	0	+5
DRIVING	22	0.2	+5
STOPPING	1	0	+5
CLOSING	8	0	-5
DONE	Until exit	0	0

Positive torque opens and holds the door while the robot passes. The robot stops before negative torque closes the door. DONE continues publishing zero velocity and torque. Nonfinite or invalid parameter values are rejected; a backward clock jump aborts the sequence when detected.

Observed result and video

Visual testing confirmed that the door opened before motion, the robot passed through without pushing it, the robot stopped beyond the doorway, and the door closed behind it. No manual door-opening command was used during the sequence.

Evidence: video/lab2_assignment3_trimmed.mp4. This 43-second clip retains seconds 14–57 of the original recording, removing only idle footage at the beginning and end. It contains no internal cuts. The initial closed door and the final closed door remain visible.

The demonstrated behavior is open loop: timing must be retuned if the starting pose, speed, or dynamics change. A completed timer sequence is not a sensor-based confirmation of success.

Assignment 4 — Configurable forward speed

Parameter flow

The `forward_speed` launch argument is converted to a floating-point ROS parameter by `ParameterValue`. `DoorController` declares the parameter and reads its `double_value` through `get_parameter_value()`. Its value sets the forward velocity during the DRIVING stage. Parameters are configured at startup and are read-only during a run.

```
ros2 launch prob_rob_labs turtlebot3_and_door_launch.py \
  run_lab2_controller:=true forward_speed:=0.3
ros2 param get /door_controller forward_speed
Double value is: 0.3
```

The parameter query returned 0.3, and the robot completed the full sequence at that setting. This verifies both launch-to-node parameter propagation and use of the parameter in the controller. Additional configurable values are `startup_delay`, `open_duration`, `drive_duration`, `stop_duration`, `close_duration`, `open_torque`, and `close_torque`.

Launch integration and source files

The top-level launch exposes `run_lab2_controller`, defaulting to false. Setting it to true includes `door_controller_launch.py` and suppresses the optional flaky door opener to prevent conflicting torque commands. Existing controller implementations, models, noise settings, and speed limits were not changed.

File under prob_rob_labs/	Role
src/door_controller/door_controller.py	ROS publishers, parameters, and timer
src/door_controller/sequence.py	Timed stages and parameter validation
launch/door_controller_launch.py	Typed launch arguments and node launch
launch/turtlebot3_and_door_launch.py	Optional top-level integration
setup.py; package.xml	Executable registration and dependency
test/test_door_sequence.py	17 automated timing/validation checks

Build and validation

```
cd "$MY_WORKSPACE"
colcon build --symlink-install --packages-select prob_rob_labs
source "$MY_WORKSPACE/install/setup.bash"
```

The package built successfully, ROS listed the new executable and launch arguments, and all 17 automated checks passed. The checks cover stage ordering, the requested speed, paused time, delayed callbacks, backward clock changes, and invalid settings. Gazebo visual tests supplied the separate evidence of successful physical traversal.

Assignment 5 — Maximum achieved velocity

Method

Three fresh simulation runs used forward_speed values of 0.3, 0.6, and 0.8 m/s. The default starting pose, world, dynamics, and 22-second driving duration were retained. The recorder was started before each launch and stopped after the sequence. It observed odometry independently; these measurements were not fed back into the controller.

```
PYTHONUNBUFFERED=1 ros2 topic echo /odom nav_msgs/msg/Odometry \
  --field twist.twist.linear.x --qos-reliability best_effort \
  | tee /tmp/lab2-speed-0.6.txt
```

For each run, the longest contiguous segment above 0.15 m/s was selected. Its first and last 20% were excluded to omit acceleration and braking. The mean of the central 60% (657 samples per trial) was used as the sustained speed. The same rule was applied to all three recordings.

Command (m/s)	Mean (m/s)	Full samples	Retained samples
0.3	0.300	2,822	657
0.6	0.500	2,060	657
0.8	0.500	1,765	657

Result

The maximum observed sustained forward velocity in the supplied simulation configuration was 0.500 m/s. Increasing the command from 0.6 to 0.8 m/s did not increase the measured velocity. This plateau agrees with the positive forward-command cap of 0.5 m/s in the supplied cmd_vel_noise_injector.

This result describes the velocity reported by /odom in the tested simulation, not an independently measured ground-truth speed or a physical TurtleBot hardware limit. Because the echo command selected only the velocity field, the saved files contain sample values without timestamps.

Submitted evidence

evidence/trimmed/speed-0.3-steady.txt, speed-0.6-steady.txt, and speed-0.8-steady.txt contain the original retained echo output without rounding, smoothing, or resampling. Full recordings are in evidence/raw/. evidence/speed_summary.csv documents statistics and the exact retained sample ranges (779–1435 inclusive, numbered from 1).

Source material

Course handout: Lab 2: ROS Nodes and Simulation Models. Implementation and model details are supported by the included source snapshot, especially the controller, launch files, door model, and supplied cmd_vel_noise_injector. Experimental evidence consists of the included screencast and ROS topic recordings.